Fly GACA — Corpus Freshness Pipeline — Design Spec

SECTION 06-operations-it TYPE spec STATUS active OWNER Founder UPDATED 2026-08-19

Status: partly built · **Maps to:** `roadmap.md` cross-cutting (AIP/AIRAC freshness), Phase 2 ingestion, Phase 10 · **Prepared:** 2026-05-30 · **Re-based on the current stack:** 2026-08-19
Companion: `strategy-competitive-teardown.md`

The competitive teardown names freshness as a structural weakness a single maintainer cannot reliably hold, and the first superseded citation as the moment our trust proposition cracks. This spec turns that liability into a **visible, automated, stamped guarantee** — the one place a funded competitor would attack, closed before they can.

The AIP changes every **28-day AIRAC cycle**; GACAR Parts amend irregularly. Today, refreshing both is semi-manual. This spec makes staleness *detectable, dated, and bounded by an SLA.*

> The pipeline is **public-data-only**. It runs on the EU VPS (`setup/setup-vps.md`, and the PDPL boundary in `hosting-facts.md`) or in CI — never against Cloud SQL. It ingests/diffs the public corpus and rebuilds indexes; it never touches personal data. Output artifacts are published to the corpus bucket; no user data crosses the boundary.

1. The two freshness clocks

Source class	Cadence	Constraint
AIP (aerodromes, charts, AIS)	every 28-day AIRAC cycle	must show effective date + "not for operational use"
GACAR Parts	irregular amendments	must track amendment number / date and re-ingest on change
Reference shelf (handbooks, ICAO refs, FAA)	rare	low-priority diff

Each artifact under `public/data/` and the RAG index (`public/data/rag-chunks.json`) carries **provenance metadata** so freshness is a property of the data, not tribal knowledge.

2. Provenance metadata (added to every corpus item)

```
{
  "source_id": "gacar-part-61",
  "source_class": "gacar",          // gacar | aip | reference
  "source_url": "https://gaca.gov.sa/...",   // canonical official copy
  "fetched_at": "2026-05-30T00:00:00Z",
  "effective": { "airac": "2026-06", "from": "2026-06-11" }, // aip only
  "amendment": "Amdt 3 (2025-11)",   // gacar only
  "content_hash": "sha256:…",        // drives change detection
  "verified_cycle": "2026-05"        // last cycle a human/CI confirmed currency
}
```

Two of these already exist in shipped form: `public/data/sources.json` is the official-source manifest (per-source `fingerprint` = the change signal, plus the AIRAC anchor), and `public/data/source-status.json` is the public freshness snapshot (`generated`, `lastChecked`, `lastUpdated`, current/next AIRAC, per-source reachability). `src/calc/library/changeTracking.ts` reads both and is what the `/updates` page renders.

3. Pipeline stages

1. **Fetch** — pull the canonical official copies for each tracked source in `sources.json`. *(Discovery/extraction is a set of agents, one per source class, emitting a normalised `records.json`.)*
2. **Hash & diff** — compare each record's `content_hash` against the persisted fingerprint. Unchanged → skip. Changed → flag for re-ingest. AIP sources are checked on the AIRAC calendar regardless of hash. **Shipped:** `npm run sync:gaca` (`scripts/sync-gaca.mjs`) is a dry run by default and reports new / changed / unchanged.
3. **Re-ingest** — merge the deltas and re-derive the indexes, incrementally rather than as a full rebuild. **Shipped:** `npm run sync:gaca:apply` merges metadata-only deltas and persists each record's provenance fingerprint; `npm run data:normalize` (`scripts/normalize-corpus-data.mjs`) normalises corpus shapes; `npm run parse:regulations` recompiles the cross-reference lookup; `npm run build:chunks` (`scripts/build-rag-chunks.mjs`) rebuilds the retrieval index `public/data/rag-chunks.json`; `npm run embeddings:upsert` refreshes the Supabase pgvector store for the dense-retrieval path. Body-bearing records (a raw PDF/eAIP asset) still need the conversion step and are reported as deferred rather than written.
4. **Stamp** — write/refresh the provenance block and bump `verified_cycle` in `source-status.json`.
5. **Validate** — run the eval harness against the rebuilt corpus; a citation that no longer resolves to a live section **fails the build** (ties to Phase 10's CI eval gate). The evals live in `ay2m/Captain-Adel` (`evals/`).
6. **Publish** — sync the refreshed `public/data/` to the corpus bucket (served **network-first** by the service worker, so a refreshed corpus reaches clients without a new app deploy), and roll a new Cloud Run revision so the API picks up the rebuilt `rag-chunks.json` — the Dockerfile

bakes it into the image so BM25 needs no cold-start fetch. Tag the revision with the AIRAC cycle.

```
fetch → hash/diff → [changed?] → re-ingest → stamp → eval-gate → publish
                        │no                                    ├─ corpus bucket
(network-first)
                        └──────────────────────────────→ skip  └─ Cloud Run revision
(rag-chunks.json)
                           (log "current")
```

> [!NOTE] **The scheduler is the missing half.** `public/data/sources.json` and `source-status.json` still name `scripts/update-sources.js` and a `.github/workflows/update-sources.yml` as their owner; neither exists in `ay2m/FlyGACA` today (the repo ships with no `.github/workflows/` at all), and `source-status.json` was last generated 2026-06-15 with `gacar` marked `unreachable`. The *app-side* half of the pipeline is real and runnable; the *automation* half is a cron job (VPS or Cloud Scheduler) that nobody has re-created since the port. That is the highest-value gap in this spec — a freshness stamp that isn't refreshed is exactly the failure §4 warns about.

4. The user-visible guarantee (the marketing surface)

- **A freshness stamp on every reader page**: "GACAR Part 61 — Amdt 3, verified AIRAC 2026-05" / "AIP AD — effective AIRAC 2026-06 · not for operational use." Reuse the existing AIRAC calculator tool's cycle logic.
- **A staleness banner**: if `verified_cycle` is older than the current AIRAC cycle for an AIP-class item, the page auto-shows "verification pending for the current cycle — confirm against the official AIP."
- **A public status line** ("Corpus current as of AIRAC YYYY-CC") on the library hub, and the `/updates` change feed for "what changed since you last looked".

This is the differentiator: *always current, verified every cycle, with the date on the page* — a claim a single manual maintainer cannot credibly make and a competitor can then market against. The automated diff + stamp + eval-gate is what makes it true rather than aspirational.

5. Captain Adel integration

- Retrieval prefers the freshest chunk; superseded chunks are excluded from `rag-chunks.json` at build time, not merely down-ranked at query time.
- The system prompt already cites the exact Part (`server/src/captain-adel-prompt.ts`); extend the `ChatSource` citation built in `server/src/corpus.ts` to carry the **amendment / AIRAC stamp** so an answer is dated, not just sourced. The chunk `lineage` block already has an `effective_date` field to hang this on.
- Grounding is decided server-side from retrieval confidence (`REFUSE_SCORE` / `GROUNDED_SCORE`), so a stale-and-thin corpus degrades into refusals rather than into confident wrong answers.

That is the fail-safe, not the goal.

- The eval harness gains a **staleness case class**: an answer citing a superseded source is a hard failure, alongside the existing citation/refusal/injection cases.

6. SLA (the commitment the pipeline lets us keep)

Source class	Refresh target
AIP-class	re-verified within **the first 72h of each new AIRAC cycle**
GACAR	re-ingested within **7 days** of a detected amendment
Reference	best-effort, monthly diff

7. What to build, in order

1. **Restore the scheduled fetch + hash/diff job** — a cron (VPS) or Cloud Scheduler trigger that runs `sync:gaca` and rewrites `source-status.json`. Cheapest, highest-value, and currently the thing blocking every claim below it.
2. Complete the provenance block across the corpus schema + backfill existing items.
3. The reader-page freshness stamp + staleness banner (user-visible win, small effort).
4. Wire incremental re-ingest (`sync:gaca:apply` → `data:normalize` → `build:chunks`) to the diff output, ending in a bucket sync + Cloud Run revision.
5. The staleness eval case + CI gate (pairs with Phase 10's CI eval work).
6. The public "current as of AIRAC" status line and the `/updates` feed hook-up.

Steps 1–3 alone convert the weakness into a marketed strength; 4–6 make the guarantee self-sustaining.